Scaled Dot-Product Attention.

The one operation every transformer is built from — derived from first principles, dissected component by component, and computed by hand on real numbers. Nothing is summarised or deferred: every matrix on the following pages is multiplied out in full.

THE EQUATION

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Worked example. A single query ("what should I watch next?") attends to four movie keys with $d_k = 3$, $d_v = 2$. Tiny on purpose — the maths is identical at scale; only the matrices grow.

Topic 1 of 5 in this module · companion to the M0.1 course page · v1.0

Read top to bottom: each step feeds the next.

Contents

1 The single equation you need

Every transformer in this track — BERT, GPT-4, LLaMA, Mistral, ViT — is built from one operation. It takes a *query*, matches it against a set of *keys*, and returns a weighted sum of *values*:

$$\boxed{\text{Attention}(Q, K, V) \;=\; \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V} \tag{1}$$

The three inputs are matrices, each a stack of row-vectors — one row per token:

Symbol	Meaning	Shape
Q (Query)	"What am I looking for?"	$(T_q,\, d_k)$
K (Key)	"What do I contain / offer?"	$(T_k,\, d_k)$
V (Value)	"What information do I pass on if chosen?"	$(T_k,\, d_v)$
Output	weighted mixture of the values	$(T_q,\, d_v)$

Here T_q is the number of queries, T_k the number of keys/values (always equal, since every key carries a value), d_k the key/query dimension, and d_v the value dimension. **Note the inner dimensions:** Q and K must share d_k so that $QK^{\top}$ is defined; K and V must share T_k so that one weight exists per value.

1.1 The four steps, always the same

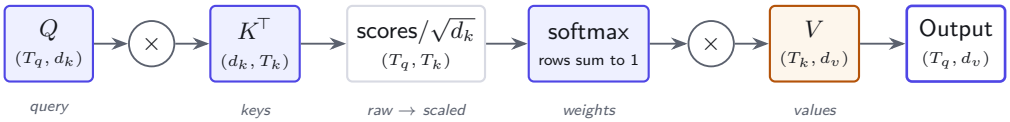
Equation (1) unfolds into exactly four operations. Every architecture in this track changes *what goes into* Q, K, V and *what mask is applied* — never the four steps themselves.

#	Operation	Produces
1	Similarity scores $S = QK^\top$	raw match of every query to every key, shape (T_q, T_k)
2	Scale $S' = S/\sqrt{d_k}$	variance-normalised scores (see Topic 2)
3	Weights $A = \mathrm{softmax}(S')$ row-wise	each row a probability distribution over keys
4	Output $O = AV$	convex combination of value rows, shape (T_q, d_v)

A dot product measures alignment: two vectors pointing the same way score high, orthogonal vectors score zero. Step 1 asks every query "how well does each key match me?". The softmax in step 3 turns those raw matches into *attention weights* that sum to 1 — a soft, differentiable lookup table. Step 4 reads out the answer by blending the values in proportion to those weights. Query = what I want; Key = what I offer; Value = what I hand over if chosen.

2 The data flow, at a glance

Before the numbers, the shape of the computation. Data flows left to right; the only place information mixes *across* tokens is the $QK^\top$ product.



Steps 1–2 produce a (T_q, T_k) score matrix — this is the only $O(T^2)$ object in the whole operation, and the reason long-context attention is expensive (Module 5). Step 4 collapses it back to (T_q, d_v).

3 The full computation — worked by hand

We take a single query ($T_q = 1$) attending to four movie keys ($T_k = 4$), with $d_k = 3$ and $d_v = 2$. These are the exact numbers from the M0.1 course page; every digit below is computed in full and is reproducible (Appendix A).

3.1 The inputs

$$Q = \begin{bmatrix} 1.0 & 0.5 & -0.3 \end{bmatrix} \ (1\times3), \qquad K = \begin{bmatrix} 0.8 & 0.3 & -0.1 \\ -0.2 & 0.9 & 0.4 \\ 1.1 & 0.6 & -0.5 \\ 0.1 & -0.3 & 0.8 \end{bmatrix} \ (4\times3), \qquad V = \begin{bmatrix} 4.2 & 0.8 \\ 1.5 & 3.7 \\ 3.9 & 1.2 \\ 0.7 & 2.1 \end{bmatrix} \ (4\times2).$$

Q is the user's "what I want next" vector; each row of K describes one movie; each row of V is the feature payload returned if that movie is chosen.

3.2 Step 1 — similarity scores $QK^\top$

Each score is the dot product of the query with one key row:

$$\begin{aligned} s_1 &= (1.0)(0.8) + (0.5)(0.3) + (-0.3)(-0.1) = 0.80 + 0.15 + 0.03 = \mathbf{0.98}, \\ s_2 &= (1.0)(-0.2) + (0.5)(0.9) + (-0.3)(0.4) = -0.20 + 0.45 - 0.12 = \mathbf{0.13}, \\ s_3 &= (1.0)(1.1) + (0.5)(0.6) + (-0.3)(-0.5) = 1.10 + 0.30 + 0.15 = \mathbf{1.55}, \\ s_4 &= (1.0)(0.1) + (0.5)(-0.3) + (-0.3)(0.8) = 0.10 - 0.15 - 0.24 = \mathbf{-0.29}. \end{aligned}$$

$$QK^\top = \begin{bmatrix} 0.98 & 0.13 & 1.55 & -0.29 \end{bmatrix}.$$

Movie 3 has the highest raw match (1.55); movie 4 is actively dissimilar (-0.29).

3.3 Step 2 — scale by $\sqrt{d_k}$

With $d_k = 3$, $\sqrt{d_k} = \sqrt{3} \approx 1.7320$. Divide every score:

$$\frac{QK^\top}{\sqrt{3}} = \begin{bmatrix} 0.5658 & 0.0751 & 0.8949 & -0.1674 \end{bmatrix}.$$

Topic 2 derives *why* the divisor is $\sqrt{d_k}$ and not d_k — it keeps the score variance near 1 so the softmax stays in its smooth, gradient-friendly regime.

3.4 Step 3 — softmax into attention weights

Softmax exponentiates each scaled score and normalises so the row sums to 1:

$$A_i = \frac{e^{s'_i}}{\sum_j e^{s'_j}}.$$

Exponentials: $e^{0.5658} = 1.7609$, $e^{0.0751} = 1.0780$, $e^{0.8949} = 2.4470$, $e^{-0.1674} = 0.8459$. Their sum is 6.1318. Dividing:

$$A = \begin{bmatrix} 0.2872 & 0.1758 & 0.3991 & 0.1379 \end{bmatrix}, \qquad \sum_i A_i = 1.0000.$$

Attention weights as a heatmap (darker = more attention):

	movie 1	movie 2	movie 3	movie 4
query	0.287	0.176	**0.399**	0.138

Scaling smooths the distribution. Without the $\sqrt{d_k}$ divisor the same scores give $\text{softmax}([0.98, 0.13, 1.55, -0.29]) = [0.287, 0.123, 0.509, 0.081]$ — noticeably more peaked on movie 3 (0.509 vs 0.399). Larger scores ⇒ sharper softmax. This is exactly the temperature effect: dividing scores by a constant flattens the weights.

3.5 Step 4 — weighted sum of the values

The output is the attention-weighted blend of the value rows, $O = AV$:

$$O = 0.2872 \begin{bmatrix}4.2\\0.8\end{bmatrix}^\top + 0.1758 \begin{bmatrix}1.5\\3.7\end{bmatrix}^\top + 0.3991 \begin{bmatrix}3.9\\1.2\end{bmatrix}^\top + 0.1379 \begin{bmatrix}0.7\\2.1\end{bmatrix}^\top$$
$$= \begin{bmatrix}1.2061\\0.2297\end{bmatrix}^\top + \begin{bmatrix}0.2637\\0.6505\end{bmatrix}^\top + \begin{bmatrix}1.5564\\0.4789\end{bmatrix}^\top + \begin{bmatrix}0.0966\\0.2897\end{bmatrix}^\top = \begin{bmatrix}\mathbf{3.1228} & \mathbf{1.6488}\end{bmatrix}.$$

The output $[3.12,\ 1.65]$ is a *convex combination* of the four value rows — dominated by movie 3 (40%) and movie 1 (29%), the two best matches. It is not a copy of any single value; it is a soft, content-addressed read of the whole memory. That single idea — "retrieve by similarity, return a weighted blend" — is the engine behind every model in this track.

4 Properties that matter

Four consequences of Equation (1) that you will rely on throughout the track.

1. **Rows are probability distributions.** Each row of A is non-negative and sums to 1, so each output row $O_i = \sum_j A_{ij} V_j$ is a *weighted average* of value rows. The output therefore lies inside the convex hull of the values — attention can interpolate between values but never extrapolate beyond them.
2. **Permutation equivariance over keys.** Reorder the rows of K and V together and the output is unchanged: attention has no built-in notion of position. That is precisely why transformers need positional encodings (Module 0.2) — without them, "I am fine" and "fine am I" look identical.
3. **Sharpness is controlled by scale.** Multiplying Q by a constant c multiplies every score by c, sharpening ($c > 1$) or flattening ($c < 1$) the softmax. In the limit $c \to \infty$ attention becomes a hard $\arg\max$ lookup; as $c \to 0$ it becomes a uniform average. This is the same knob as the temperature in sampling.
4. **Differentiable end to end.** Every step — matmul, divide, exp, normalise, matmul — is smooth, so gradients flow back through the weights into whatever produced Q, K, V. Attention *learns what to look for*; nothing about the lookup is hand-coded.

d_k is the *per-head* key dimension, not d_{model}. In multi-head attention (Topic 4) the model dimension is split across h heads, so the divisor is $\sqrt{d_{\text{model}}/h}$, e.g. $\sqrt{64} = 8$ for BERT-base ($d_{\text{model}} = 768,\ h = 12$). Scaling by $\sqrt{d_{\text{model}}}$ instead is a common and silent bug.

5 Where this sits in the track

Equation (1) is the atom. The remaining topics in this module change one piece each, leaving the four steps intact:

Topic	What it changes	Why
2	the divisor $\sqrt{d_k}$	keep softmax in its smooth regime
3	the softmax step itself	turn scores into a valid distribution; its gradient
4	*what goes into* Q, K, V	h parallel heads on d_{model}/h subspaces
5	a mask added before softmax	bidirectional (BERT) vs causal (GPT)

The full attention used in practice is therefore

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V,$$

with M the mask of Topic 5. Master this one operation and the rest of the track is variations on a theme.

A Every number used (reproducible)

For full reproducibility, the inputs and all intermediate results:

$$Q = \begin{bmatrix} 1.0 & 0.5 & -0.3 \end{bmatrix}, \quad K = \begin{bmatrix} 0.8 & 0.3 & -0.1 \\ -0.2 & 0.9 & 0.4 \\ 1.1 & 0.6 & -0.5 \\ 0.1 & -0.3 & 0.8 \end{bmatrix}, \quad V = \begin{bmatrix} 4.2 & 0.8 \\ 1.5 & 3.7 \\ 3.9 & 1.2 \\ 0.7 & 2.1 \end{bmatrix}.$$

$$\begin{aligned} \text{raw scores } QK^\top &= [\,0.98,\ 0.13,\ 1.55,\ -0.29\,] \\ \text{scaled } QK^\top/\sqrt{3} &= [\,0.5658,\ 0.0751,\ 0.8949,\ -0.1674\,] \\ \text{weights } A = \text{softmax}(\cdot) &= [\,0.2872,\ 0.1758,\ 0.3991,\ 0.1379\,] \quad (\textstyle\sum = 1) \\ \text{output } O = AV &= [\,3.1228,\ 1.6488\,] \end{aligned}$$

```
import numpy as np
Q = np.array([[1.0, 0.5, -0.3]])
K = np.array([[0.8,0.3,-0.1],[-0.2,0.9,0.4],[1.1,0.6,-0.5],[0.1,-0.3,0.8]])
V = np.array([[4.2,0.8],[1.5,3.7],[3.9,1.2],[0.7,2.1]])
s = Q @ K.T / np.sqrt(3)
A = np.exp(s - s.max()); A /= A.sum()
print(A, A @ V)   # [[0.2872 0.1758 0.3991 0.1379]]  [[3.1228 1.6488]]
```

Marevlo Research · Transformer Track · Module 0.1 “Attention Refresher” · Topic 1 of 5: Scaled Dot-Product Attention.